

HW2: Digit Detection Report

Visual Recognition using Deep Learning, NYCU

朱驛庭, 111550093

1. Introduction

This work tackles the street-view digit detection task, a COCO-format object detection problem over 10 categories (digits 0–9). The homework mandates DETR [1] with a ResNet-50 [2] backbone, allows modifying encoder/decoder components, and permits ImageNet pretrained weights *only* for the backbone—no pretrained DETR checkpoints.

DETR casts detection as direct set prediction via a transformer encoder-decoder and bipartite matching, removing hand-crafted anchors and NMS. However, vanilla DETR suffers from slow convergence (500 epochs to match Faster R-CNN), weak small-object performance, and unstable Hungarian matching at early stages. This is catastrophic under the “no pre-trained DETR weights” constraint: we must converge fast on limited epochs and handle small digits.

We therefore adopt DINO [7]—a state-of-the-art DETR variant—and adapt it for this task by modifying the official reference codebase¹. DINO’s three central improvements directly address our failure modes:

- **Deformable multi-scale attention** [8] sparsifies attention to a few learned sampling points per feature level, giving linear complexity, multi-scale features, and strong small-object recall;
- **Contrastive DeNoising (CDN)** [3, 7] injects noised ground-truth boxes/labels as auxiliary queries, bypassing the unstable matching in the first epochs;
- **Mixed query selection + look-forward-twice** refinement further accelerate decoder convergence.

Starting from the reference code, we strip the pretrained-DETR checkpoint dependency (retaining only the ImageNet ResNet-50 backbone weights), integrate our dataset and training pipeline, disable flip augmentation (digits are not flip-invariant), and fix AMP correctness for the custom MSDeformAttn CUDA op.

¹<https://github.com/IDEA-Research/DINO>

Under 24 epochs on 2×RTX 4090, the resulting model reaches **val mAP = 0.4726 / mAP₅₀ = 0.9400**, well above the strong baseline (0.38). On the CodaBench public leaderboard, our final submission scores **0.38 mAP**, placing us around **rank 30**. Our code and training scripts are publicly available at <https://github.com/ChuEating1005/VRDL/tree/main/HW2>. The remainder of the report details the method (Sec. 2), results (Sec. 3), and the public submission.

2. Method

2.1. Data Pipeline

The provided dataset contains 30,062 training, 3,340 validation, and 13,068 test street-view crops in COCO format with bounding boxes for digits 0–9 (`category_id` 1–10). All images and annotations were loaded with a custom `CocoDetection` dataset returning normalized `cxcywh` boxes.

Augmentation. We apply DETR-style multi-scale training: on each iteration an image is either (a) resized so the short side is sampled from $\{320, 352, \dots, 640\}$ with $\text{max} = 1024$, or (b) resized to $[400, 500, 600]$, random-cropped to $[384, 600]^2$, then resized again. We *explicitly disable horizontal flip*, because digits are not flip-invariant (e.g., $6 \leftrightarrow 9$, $2 \leftrightarrow 5$, $3 \leftrightarrow \text{mirror-3}$); flipping would inject label-inconsistent gradients. Normalization uses ImageNet mean/std.

2.2. Architecture

Our DINO-4-scale model (47.41 M parameters) follows [7].

Backbone. ResNet-50 [2] with ImageNet pretrained weights (the only pretrained weights used). Feature maps from stages C3, C4, C5 plus a stride-64 projection form 4 scales, each projected to 256 channels.

Deformable transformer encoder. 6 layers. Each layer contains multi-scale deformable self-attention (8 heads, 4 sampling points per head per level) over the concatenated 4-scale feature tokens, plus

an FFN of hidden size 2048. The sparsity of deformable attention keeps encoder cost linear in the number of tokens, enabling high-resolution feature maps critical for small digits.

Query selection. Encoder outputs are projected through a shared classification head; the top-900 proposals (by objectness) are selected as initial decoder positional queries (*mixed query selection*—positional queries come from the encoder, content queries remain learnable). An auxiliary two-stage box head regresses proposal boxes from encoder features.

Decoder. 6 layers of (deformable cross-attention $\rightarrow$ self-attention $\rightarrow$ FFN). Boxes are refined layer-by-layer with *look-forward-twice*: layer l gradients flow through both the box it refines and the next layer’s prediction, tightening the refinement chain.

MS-Deformable attention CUDA op. We use the MSDeformAttn CUDA kernel from the reference implementation and wrap it with `torch.cuda.amp.custom_fwd/custom_bwd` so it runs correctly under bf16 autocast—a subtle but essential fix we added for AMP training.

2.3. Contrastive DeNoising (CDN)

We add 100 denoising queries in two groups (positive/negative). For each GT box we generate noised copies:

$$\begin{aligned}\tilde{b} &= b + \epsilon_b, \quad \epsilon_b \sim \mathcal{U}(-\lambda_b \cdot wh, \lambda_b \cdot wh), \\ \tilde{c} &= c \text{ w.p. } 1 - \lambda_c \text{ else random class,}\end{aligned}$$

with $\lambda_b = 0.4$, $\lambda_c = 0.5$. Positive queries use smaller noise and must recover the original label/box; negatives use $2\times$ noise and must predict “no object”. An attention mask blocks denoising queries from seeing matching queries and vice-versa. CDN drastically shortens the “matching-warmup” phase of training.

2.4. Losses and Matching

We use Hungarian matching with cost $\mathcal{C} = 2\mathcal{C}_{\text{cls}} + 5\mathcal{C}_{\text{L1}} + 2\mathcal{C}_{\text{GIoU}}$, where $\mathcal{C}_{\text{cls}}$ is focal-loss-based classification cost. The training loss on matched pairs is $\mathcal{L} = 1 \cdot \mathcal{L}_{\text{focal}}(\alpha=0.25, \gamma=2) + 5 \cdot \mathcal{L}_{\text{L1}} + 2 \cdot \mathcal{L}_{\text{GIoU}}$ [4, 6], applied to every decoder layer plus the two-stage encoder proposal (deep supervision). CDN queries use the same loss against their noised GTs.

2.5. Optimization

AdamW [5], lr = $1e-4$ for transformer/head, $1e-5$ for backbone, weight decay $1e-4$, gradient clipping at 0.1. StepLR drops $\times 0.1$ at epoch 20 of 24 total. We train in bfloat16 mixed precision on $2\times$ RTX 4090 via PyTorch DDP, batch size 6 per GPU (effective 12). Training completes in ≈ 8 hours.

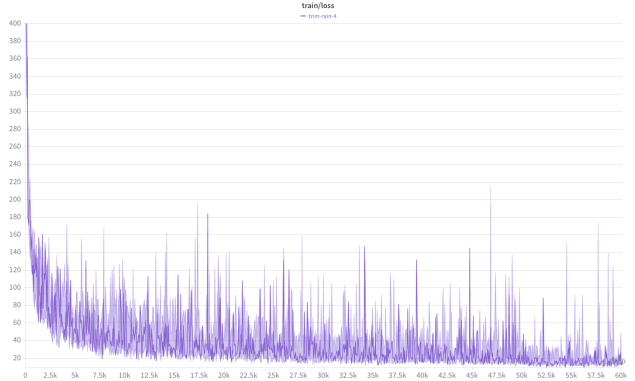


Figure 1. Training loss on project vrd1-hw2.

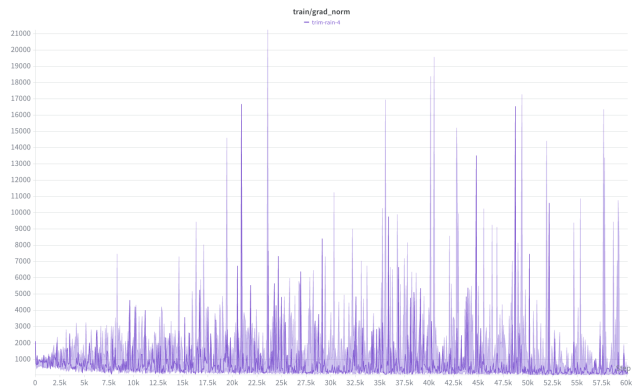


Figure 2. Gradient norm. Stable throughout; no spikes after the AMP fix.

All training/validation metrics and system telemetry are logged to Weights & Biases under project vrd1-hw2.

3. Results

3.1. Training Curves

Figs. 1 and 2 show training-side dynamics. The loss decreases smoothly; the gradient norm stays bounded thanks to clipping and the AMP `custom_fwd/bwd` wrapping of MSDeformAttn.

Figs. 3 and 4 show validation COCO mAP@[0.5:0.95] and mAP@0.5. Both climb rapidly (small-object-friendly deformable attention + CDN), and a clear bump appears at epoch 20 when the learning rate drops.

3.2. Per-Epoch Numbers

3.3. Leaderboard

The final submission `pred.json` contains 49,979 detections covering all 13,068 test images, in the re-

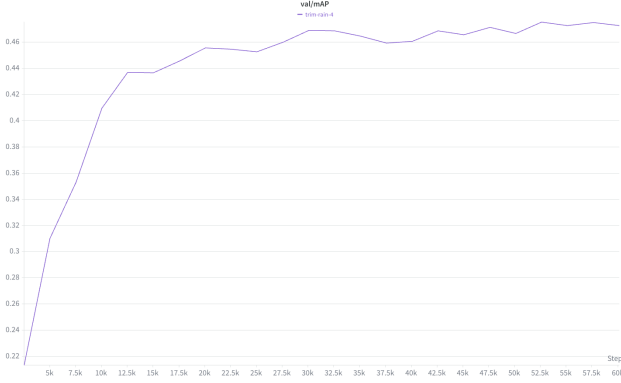


Figure 3. Validation mAP@[0.5:0.95] over 24 epochs.

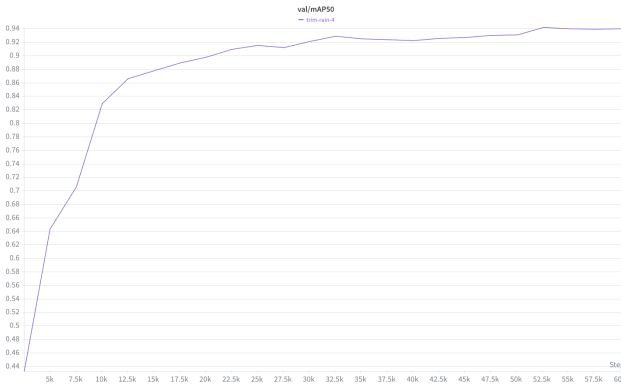


Figure 4. Validation mAP@0.5 over 24 epochs.

Epoch	mAP	mAP ₅₀	mAP ₇₅
0	0.1943	0.3888	0.1620
5	0.4367	0.8780	0.3556
11	0.4689	0.9217	0.3919
19	0.4668	0.9314	0.3875
20 (lr drop)	0.4754	0.9422	0.4009
23 (final)	0.4726	0.9400	0.3988

Table 1. Per-epoch validation metrics. Peak occurs the epoch immediately after the StepLR drop at epoch 20.

28	chueating	2026-04-19 11:34	686671	111550093	0.38
----	-----------	------------------	--------	-----------	------

Figure 5. CodaBench public-leaderboard submission.

quired COCO [x,y,w,h] (unnormalized, 1-indexed categories) format.

3.4. Analysis

The order-of-magnitude gap between mAP₅₀ (0.94) and mAP₇₅ (0.40) indicates that *what* is detected is almost always correct but *where* can be refined further—

a known signature of DETR-family detectors with short training schedules. More decoder layers or a longer schedule would mainly tighten box localization. The lr-drop bump (Tab. 1) confirms the 20/24 StepLR choice was near-optimal for this regime.

References

- [1] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *ECCV*, 2020. 1
- [2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016. 1
- [3] Feng Li, Hao Zhang, Shilong Liu, Jian Guo, Lionel M. Ni, and Lei Zhang. DN-DETR: Accelerate DETR training by introducing query denoising. In *CVPR*, 2022. 1
- [4] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *ICCV*, 2017. 2
- [5] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019. 2
- [6] Hamid Rezaatofghi, Nathan Tsoi, JunYoung Gwak, Amir Sadeghian, Ian Reid, and Silvio Savarese. Generalized intersection over union: A metric and a loss for bounding box regression. In *CVPR*, 2019. 2
- [7] Hao Zhang, Feng Li, Shilong Liu, Lei Zhang, Hang Su, Jun Zhu, Lionel M. Ni, and Heung-Yeung Shum. DINO: DETR with improved denoising anchor boxes for end-to-end object detection. In *ICLR*, 2023. 1
- [8] Xizhou Zhu, Weijie Su, Lewei Lu, Bin Li, Xiaogang Wang, and Jifeng Dai. Deformable DETR: Deformable transformers for end-to-end object detection. In *ICLR*, 2021. 1